DELHI TECHNICAL CAMPUS
GREATER NOIDA

PROJECT REPORT

ON

EV CHARGING STATION MANAGEMENT SYSTEM

Submitted in partial fulfilment of the requirements for the subject

OOPS USING C++

Submitted to

Ms. Kajal Kumari

Submitted by

S. No.	Student Name	Enrollment Number	Branch
1	Ronit Sharma	01018015625	CSE (AI & ML)
2	Bhumika	00818015625	CSE (AI & ML)
3	Sarthak Kumar	05318015625	CSE (AI & ML)
4	Aditi Raj	11318015625	CSE (AI & ML)

Academic Session: 2026–27

DECLARATION

We hereby declare that the project report entitled **"EV Charging Station Management System"**, submitted for the subject **OOPS USING C++** at Delhi Technical Campus, Greater Noida, describes the work undertaken by our team under the guidance of **Ms. Kajal Kumari**.

The report explains the object-oriented design, charging algorithms, implementation, customer interface and testing of the project. The implementation descriptions are based on the accompanying source files. External libraries, frameworks and project resources used in the system are identified in the report.

The system is an educational simulation. It demonstrates charging decisions and software workflows without operating physical charging equipment or collecting real payments. Features prepared for future integration are distinguished from the currently implemented local functionality.

We acknowledge the contributions of all team members and the use of open-source software in the development and presentation of this project.

Student Name	Enrollment Number	Signature
Ronit Sharma	01018015625	_____
Bhumika	00818015625	_____
Sarthak Kumar	05318015625	_____
Aditi Raj	11318015625	_____

Date: _____

Place: Greater Noida

ACKNOWLEDGEMENT

We express our sincere gratitude to **Ms. Kajal Kumari** for her guidance and support during the development of our OOPS USING C++ project, “EV Charging Station Management System”. Her guidance helped us relate classroom concepts to the design of a modular software application.

We thank the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), Delhi Technical Campus, Greater Noida, for providing the academic environment and resources required to study and implement the project.

The project provided an opportunity to apply classes, encapsulation, constructors, destructors, function overloading, templates and exception handling to a practical management problem. It also helped us understand how a C++ domain model can be connected to a browser interface, persistent storage and simulated external events.

We acknowledge the developers and maintainers of the open-source tools used in the project, including the C++ toolchain, Crow, Asio, SQLite, Python, Flutter and the mapping libraries and datasets.

Finally, we thank our teammates for their cooperation in discussing the requirements, developing the application, checking its behavior and preparing this report.

Ronit Sharma | Bhumika | Sarthak Kumar | Aditi Raj

ABSTRACT

The growing use of electric vehicles creates a need for software that can coordinate charging requests, charger compatibility, available time, energy cost and renewable power preferences. The EV Charging Station Management System explores these decisions through a modular C++17 simulation and a supporting full-stack demonstration.

The core models vehicles, users, chargers, sessions and charging stations using encapsulated classes. A range-safety service estimates whether a vehicle can reach a station with a safety buffer. A time-bound scheduler selects a compatible charger and calculates the power needed to meet a requested deadline. A pricing engine demonstrates function overloading through flat-rate, time-of-use and solar-discount calculations. Queue components prioritize requests using emergency status, battery level, bookings, compatibility, service duration and waiting time.

The system also demonstrates solar/grid source selection and an emergency override that pauses an eligible non-emergency session and calculates compensation. Generic programming is represented by a policy-based priority queue and an event-logging function template. Invalid operations are handled through exceptions and structured API errors.

The Pluggo customer web application connects to a Crow API with SQLite persistence for visitor preferences and charging-session records. A Python WebSocket simulator supplies charging updates. Legacy operator routes and the standalone Flutter demonstration remain separate components. PostgreSQL and Keycloak resources are integration scaffolding rather than the active customer database and login system.

This report presents requirements, architecture, OOP concepts, algorithms, complexity analysis, worked calculations, implementation details, screenshots and verification. The project demonstrates software coordination; it does not control physical chargers or process real payments.

Keywords

C++17, object-oriented programming, EV charging, priority queue, templates, scheduling, SQLite, Crow, charging simulation.

TABLE OF CONTENTS

Section	Page
Declaration	2
Acknowledgement	3
Abstract	4
1. Introduction	6
2. Problem Statement and Objectives	7
3. Background and OOP Concepts	8
4. Methodology and System Architecture	9
5. Tools, Technologies and Data Storage	10
6. Algorithms and Charging Rules	11
7. Complexity Analysis and Worked Example	12
8. Implementation Guide	13
8. Implementation: Interface Demonstration	14
9. Testing and Results	15
10. Applications, Limitations and Future Scope	16
11. Conclusion	16
12. References	17
Appendix A: Demonstration Evidence	17
Appendix B: Code Extracts and Pseudocode	18

Figures: 1. System architecture (page 9); 2. Pluggo customer home (page 14); 3. Charging session interface (page 14); 4. Session history (page 17).

1. INTRODUCTION

1.1 Background

Electric vehicles depend on charging infrastructure with different connectors, power ratings and occupancy conditions. A driver may need to reach a station safely, charge before departure and remain within a fixed budget. A station operator must coordinate competing requests while considering emergency needs and renewable energy availability.

These concerns provide a practical setting for object-oriented programming. A vehicle owns battery and connector information; a charger owns power and availability information; a session records a charging interaction. Separate services can apply scheduling and pricing rules without combining every responsibility into one large function.

1.2 Project Overview

The project begins with a C++17 domain engine and an interactive console demonstration. A Crow server exposes JSON APIs for the legacy operator demonstration and a separate set of customer routes. Pluggo is the customer browser application, with station discovery, vehicle preferences, simulated charging, session history and zero-money receipts.

1.3 Motivation

- Connect OOP concepts to a recognizable management problem.
- Make charging constraints and queue decisions visible through examples.
- Separate domain rules from interfaces and storage responsibilities.
- Demonstrate validation and repeatable testing rather than only successful inputs.

1.4 Scope

The scope includes software simulation, compatible-charger selection, deadline planning, budget calculations, priority ordering, solar/grid decisions and emergency preemption. The connected customer application adds local persistence and simulated meter events. Live utility tariffs, authenticated production accounts, real payment settlement and physical EVSE integration are outside the implemented demonstration.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Design a modular system that represents EV charging entities and processes charging requests under connector, time, budget and power constraints. The system should explain why a request is accepted, limited or unable to meet its requested deadline, while maintaining understandable state and repeatable behavior.

2.2 Objectives

- Model domain entities using encapsulated C++ classes.
- Demonstrate construction, copying, destruction, overloading and templates.
- Prioritize requests and preserve deterministic ordering for ties.
- Support range evaluation, pricing, source selection and emergency handling.
- Present customer workflows with persistent simulated session records.

2.3 Functional Requirements

ID	Requirement	Expected behavior
FR-01	Range and deadline	Evaluate buffered reachability and estimate a compatible charging plan.
FR-02	Budget and pricing	Calculate tariffs and limit energy to the remaining budget.
FR-03	Queue management	Rank requests; reject empty-queue removal.
FR-04	Energy and emergency	Choose a simulated source; preempt only an eligible normal session.
FR-05	Customer workflow	Discover, verify, start, stop and settle a simulated session.
FR-06	Persistence	Restore visitor preferences and session history from SQLite.

2.4 Non-functional Requirements

Correctness: respect connector and vehicle power limits. **Clarity:** keep classes and service responsibilities distinct. **Robustness:** reject invalid inputs and duplicate conflicting requests. **Maintainability:** keep the C++ domain usable independently of the browser interface.

3. BACKGROUND AND OOP CONCEPTS

3.1 Classes, Objects and Encapsulation

Vehicle, User, Charger, Session and ChargingStation describe the main entities. Private data members prevent callers from directly changing internal values. Accessors expose relevant information, while operations such as Vehicle::setSocPercent validate changes. Session contains User and Vehicle objects; ChargingStation contains a collection of Charger objects. These are examples of composition.

3.2 Constructors, Copying and Destructors

The entity classes declare default, parameterized and copy constructors. Default constructors delegate to valid initial values. Parameterized constructors establish the object state and reject invalid data. Copy constructors and destructors are explicitly declared; many definitions use = **default**, relying on the normal value semantics of strings and containers. The design does not require manual memory allocation to demonstrate object lifetime.

3.3 Overloading and Generic Programming

PricingEngine::calculateCost has overloads with two, three and four double parameters. They represent flat pricing, solar-discount pricing and time-of-use pricing. Overload selection happens at compile time. SmartPriorityQueue<T, ScorePolicy> separates queue storage from its scoring policy; logEvent<T> accepts different printable value types.

Concept	Evidence in the implementation
Abstraction	RangeSafetyService and TimeBoundScheduler expose focused operations.
Composition	Session contains User and Vehicle; station contains chargers.
Compile-time polymorphism	Function overloading and class/function templates.
Exception handling	invalid_argument for invalid values; underflow_error for empty queues.

3.4 Design Boundary

The inspected core does not use an inheritance hierarchy or virtual methods to model charger types. Connector and charger categories are represented using enum classes. Therefore, runtime polymorphism and inheritance are not claimed as implemented features.

4. METHODOLOGY AND SYSTEM ARCHITECTURE

4.1 Development Methodology

Identify entities and constraints, implement independent domain services, exercise them through the console, expose API operations, connect the customer interface, persist customer state and verify normal and boundary cases. This sequence keeps the OOP engine understandable before introducing web integration.

4.2 Logical Architecture

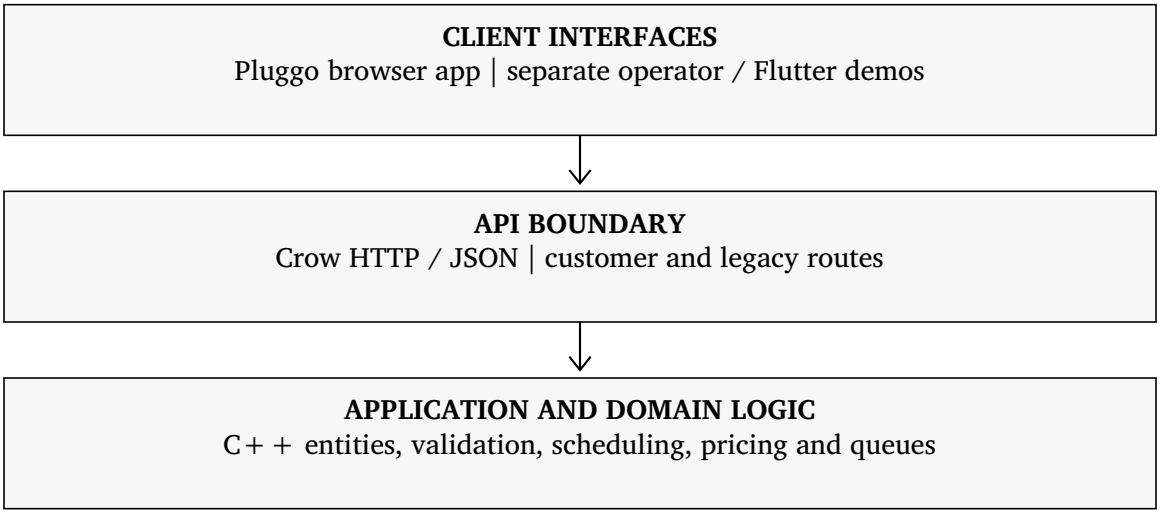


Figure 1. Logical architecture. Customer charging updates enter through the simulator/API boundary; legacy algorithms are also exposed separately.

4.3 Data and Event Flow

The browser restores a visitor snapshot, verifies a connector and requests a session. The API validates the request and persists it in SQLite. Simulated meter events update energy within target and budget limits. Stop and checkout create an ended session and a stable demo receipt.

4.4 Component Boundaries

Customer storage uses SQLite; legacy operator sessions remain in memory. Flutter is a separate demo. PostgreSQL and Keycloak files provide scaffolding for future integration.

5. TOOLS, TECHNOLOGIES AND DATA STORAGE

5.1 Technology Selection

Technology	Role
C++17; standard library	Domain classes, algorithms, containers and exceptions.
CMake; CTest; clang++	Build configuration and automated feature checks.
Crow + standalone Asio	HTTP/JSON API and WebSocket integration.
HTML, CSS and JavaScript	Pluggo browser experience and operator interface.
SQLite	Implemented local customer persistence.
Python simulator	OCPP-shaped charging events over WebSocket.
Leaflet / OpenStreetMap	Map display and public station directory snapshot.
Flutter; Docker; OpenAPI	Separate mobile demo, deployment packaging and API contracts.

5.2 Implemented Data Model

The customer_visitors table stores a visitor ID, selected vehicle and saved-station JSON. The customer_sessions table stores session ID, owner, idempotency key, request, connector, timestamps, settlement flag and a JSON payload containing session details. A unique constraint on (owner, idem) supports safe request retries.

The customer request wrapper serializes operations with a mutex and uses SQLite transactions. Owner checks keep visitor sessions separate. This is local visitor identity through a cookie, rather than a recovered account with cross-device login.

5.3 Feasibility and Deployment

The console needs a C++17 compiler. The connected local application additionally needs SQLite development support, Git, Python and web dependencies. No dedicated charging hardware is required. Static hosting serves the interface but cannot execute the C++ API; the connected experience requires a backend host and simulator. PostgreSQL, Keycloak and container configuration are supporting resources with separate integration and validation needs.

6. ALGORITHMS AND CHARGING RULES

6.1 Buffered Range Evaluation

Usable energy = battery capacity × current SoC / 100. Available range = usable energy / consumption per kilometre. Required range = station distance × (1 + buffer / 100). The station is considered reachable when available range is at least the required buffered distance.

6.2 Deadline-Based Scheduling

Energy needed = capacity × (target SoC − current SoC) / 100. Required input power = energy needed / (available hours × charging efficiency). The scheduler scans available compatible chargers and limits each to the smaller of charger power and vehicle acceptance power. It selects the smallest sufficient capacity and throttles assigned power to the requirement. If no charger meets the deadline, it returns the fastest compatible option with deadlineFeasible set to false.

6.3 Priority Scoring

Condition	Score contribution
Emergency request or vehicle	+1000
SoC below 10%; otherwise below 20%	+300; otherwise +150
Pre-booked request	+100
Compatible charger available / unavailable	+40 / −200
Shorter estimated service	max(0, 60 − minutes) × 0.25
Waiting time	min(120, non-negative whole minutes) × 0.5

The template queue computes the score at insertion and breaks exact ties by insertion sequence. QueueManager instead recalculates scores when ordering its vector, using arrival time to break ties. The two components demonstrate different approaches to changing priorities.

6.4 Solar and Emergency Decisions

Full live solar supply receives a 15% illustrative discount; partial solar or stored energy selects grid fallback with 7.5%. Emergency override scans charging non-emergency sessions and their chargers, selecting the highest-rated compatible candidate, pausing it and returning an INR 50 compensation credit. A non-emergency request or missing eligible candidate is rejected.

7. COMPLEXITY ANALYSIS AND WORKED EXAMPLE

7.1 Complexity Summary

Let C be the charger count, N the queued request count and S the active-session count. Scoring one request uses a fixed number of arithmetic operations.

Operation	Time	Additional / stored space
Range, pricing, solar decision	O(1)	O(1)
Deadline scheduler / charger lookup	O(C)	O(1) working space
Template queue push / pop	O(log N)	O(N) stored nodes
Template queue top	O(1)	O(1) working space
QueueManager orderedRequests	O(N log N)	O(N) copied requests
QueueManager serveNext	O(N log N)	O(N) ordered copy
Emergency candidate selection	O(S × C)	O(1) working space

7.2 Worked Charging Example

Use the domain test vehicle: 60 kWh battery, 20% initial SoC, 0.2 kWh/km consumption, CCS2 connector and 100 kW vehicle limit. Usable energy is 12 kWh, giving 60 km range. At a 20% safety buffer, a 50 km trip requires 60 km and is accepted; 51 km requires 61.2 km and is rejected.

Charging to 80% requires $60 \times (80 - 20) / 100 = \mathbf{36\ kWh}$. With one hour available and 90% efficiency, required input power is $36 / 0.9 = \mathbf{40\ kW}$. A compatible 60 kW charger is sufficient, so the scheduler chooses it and assigns 40 kW.

With only 0.1 hours, the requirement rises to 400 kW. The 150 kW charger is limited by the vehicle to 100 kW, giving $36 / (100 \times 0.9) = \mathbf{0.4\ hours}$. The returned plan therefore cannot meet the requested deadline.

7.3 Pricing and Budget

20 kWh at INR 18/kWh costs INR 360. A 15% solar discount reduces it to INR 306. For an INR 300 budget at INR 20/kWh, 15 kWh is affordable in total; if 5 kWh is already delivered, 10 kWh remains. These are deterministic model calculations, not measured charger performance.

8. IMPLEMENTATION GUIDE

8.1 Source Organization

File / directory	Responsibility
include/ev_system.hpp	Entity declarations, service interfaces and queue template.
src/ev_system.cpp	Validation, scheduling, pricing, queue and energy rules.
src/main.cpp	Interactive console and automated demonstration.
src/api_server.cpp	Crow server and legacy operator endpoints.
src/customer_api.cpp	Customer API, SQLite storage and simulator boundary.
frontend/; ocpp-simulator/	Browser interfaces and Python simulated events.
tests/ev_tests.cpp; tests/api/	Domain checks and customer integration verification.

8.2 Build and Run

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
./build/ev_charging
```

The console offers deadline planning, budget charging, queue operations, solar simulation, emergency override and a complete automated demonstration. Platform-specific executable paths may vary.

```
python3 -m pip install -r ocpp-simulator/requirements.txt
python3 tools/run-local.py --build
```

The local customer launcher starts the browser proxy on port 4173, the C++ API on 18080 and the charging simulator. Open http://127.0.0.1:4173/#home. The API build requires SQLite support and Crow/Asio dependencies.

8.3 Customer Request Lifecycle

Bootstrap establishes a visitor cookie. Session creation requires the application request header and an idempotency key. The server validates connector, vehicle, target and budget; it owns energy and pricing calculations. Stop and checkout are retry-safe. Checkout records a simulated receipt and collects INR 0. Network failures show a retry state; standalone browser simulation is selected explicitly using ?demo=1.

8. IMPLEMENTATION: INTERFACE DEMONSTRATION

The following saved project screenshots show the connected customer interface using simulated charging data.

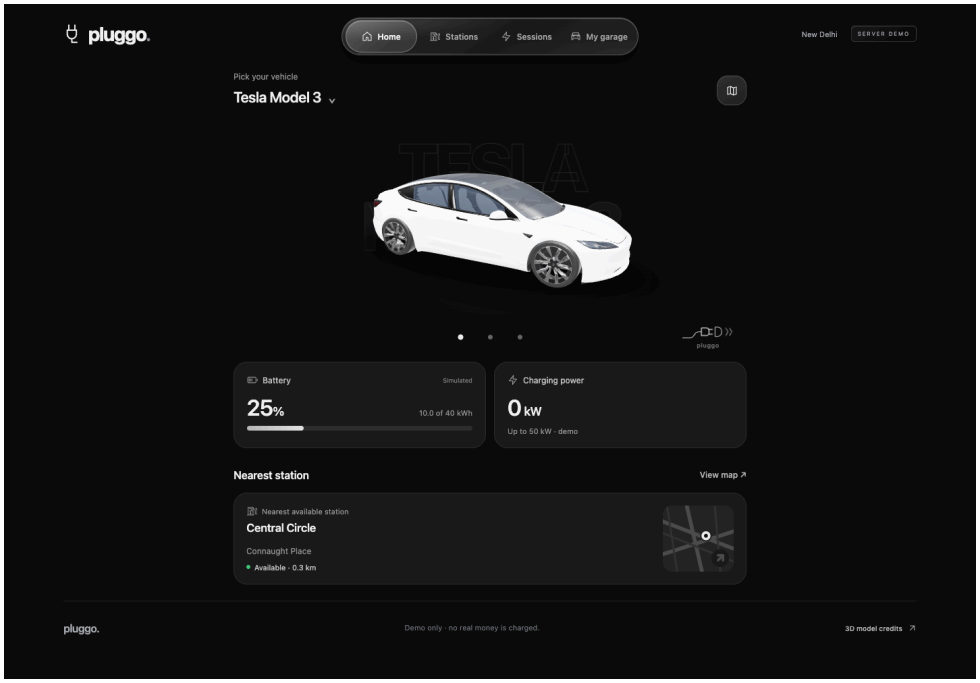


Figure 2. Pluggo home: selected vehicle, battery status and nearest station.

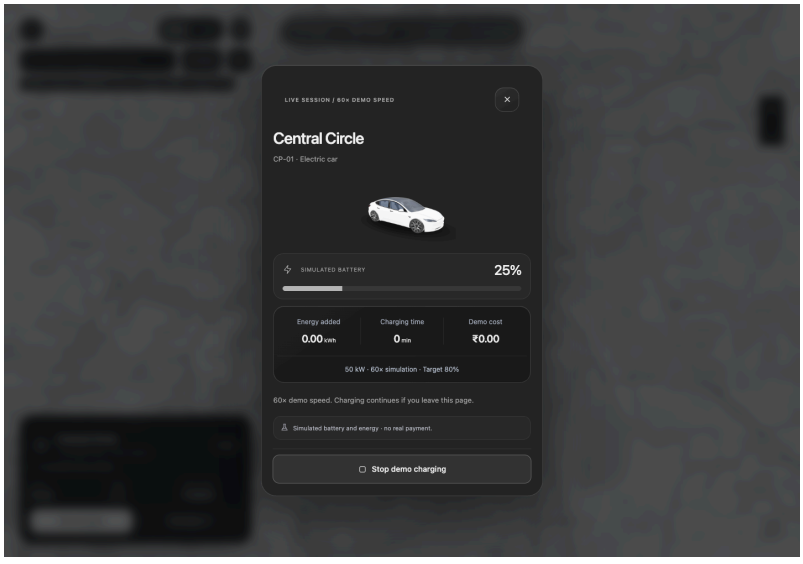


Figure 3. Simulated charging session with target, battery, energy and stop action. Screenshots are existing project evidence, not physical charger measurements.

9. TESTING AND RESULTS

9.1 Verification Approach

The C++ feature executable was compiled from the inspected source using clang++ with C++17 and warning flags. It completed successfully and printed: **"All EV feature checks passed."** The checks use deterministic inputs and compare calculated values or expected state transitions.

Test group	Expected / verified domain behavior
Range boundary	50 km accepted; 51 km rejected with the test safety buffer.
Deadline and compatibility	40 kW plan for one hour; impossible deadline flagged; incompatible connector rejected.
Pricing and budget	Flat 360; time-of-use 290; solar 306; remaining allowance 10 kWh.
Priority and ties	Emergency before booked before ordinary; FIFO for equal template scores.
Empty queue	Removal throws underflow_error.
Solar selection	Full solar, partial-solar fallback and night grid cases distinguished.
Emergency override	Normal session paused; charger transferred; credit returned; invalid preemption rejected.

9.2 Customer Integration Verification

The repository contains a real-process C++/SQLite API suite covering restart persistence, visitor isolation, origin/request-header checks, server validation, idempotency, connector contention, target caps, receipt replay and authenticated WebSocket simulator metering. The verification status for this report is recorded below.

Passed during report preparation using the existing local C++ API build: persistence, isolation, request validation, idempotency, connector races, target caps, receipt replay and authenticated simulator metering.

9.3 Interpretation

Passing checks support the tested local software behaviors. They do not demonstrate production throughput, complete OCPP conformance, physical charging safety, real vehicle telemetry or payment processing. The report includes no invented latency, memory benchmark or charging-efficiency measurement.

10. APPLICATIONS, LIMITATIONS AND FUTURE SCOPE

10.1 Applications

- Classroom demonstrations of C++ OOP and generic programming.
- Exploration of deadline, budget and compatibility constraints.
- Comparison of static heap scores and recalculated request ordering.
- Prototype customer journeys for station discovery and simulated charging.

10.2 Current Limitations

The charging model uses simplified constant-power calculations and does not reproduce battery tapering, thermal behavior or grid control. The solar manager chooses a source and discount; it is not a complete interval-by-interval storage dispatch model. Emergency handling changes session state and returns a credit; automated resumption, financial settlement and a persisted fairness ledger require additional implementation.

Customer identity is visitor-cookie based. Legacy operator sessions remain in memory. The public station map is a directory snapshot rather than a live network inventory. The Flutter demonstration and legacy routes are separate from the persistent customer browser workflow. Physical hardware, production identity and real payments are outside the implemented scope.

10.3 Future Scope

- Integrate production accounts, JWT validation and recovery.
- Evaluate PostgreSQL migration and multi-instance coordination.
- Add audited charger connectivity and richer charging curves.
- Persist emergency compensation, resumption and queue events.
- Measure performance with repeatable, documented workloads.

11. CONCLUSION

The project connects encapsulated C++ entities, overloaded pricing functions, policy-based templates and exception handling to an EV charging management problem. Its domain rules are demonstrated through a console and supporting APIs, while Pluggo adds a persistent customer simulation. The result is an inspectable educational platform whose current behavior and future integrations can be explained separately.

12. REFERENCES

Implementation sources inspected for this report are from the accompanying project checkout. Paths below identify the relevant evidence; they do not imply external performance or hardware certification.

[1] Team project repository: github.com/Bhumika-gaba23/EV-charging-management.

[2] include/ev_system.hpp and src/ev_system.cpp — domain entities, services, queue policy and algorithms.

[3] src/main.cpp and tests/ev_tests.cpp — console workflow and domain verification.

[4] src/customer_api.cpp and docs/customer-backend.md — customer identity, SQLite persistence and simulator behavior.

[5] CMakeLists.txt and openapi-customer.yaml — build dependencies and customer API contract.

[6] tests/api/customer_integration.py and docs/ui-audit/backend-connected/ — integration checks and saved UI evidence.

APPENDIX A: DEMONSTRATION EVIDENCE

For a presentation, demonstrate vehicle selection, station verification, session start, simulated charging, stop, zero-money checkout and session history. Use the console separately to explain priority scoring, deadline scheduling and emergency preemption.

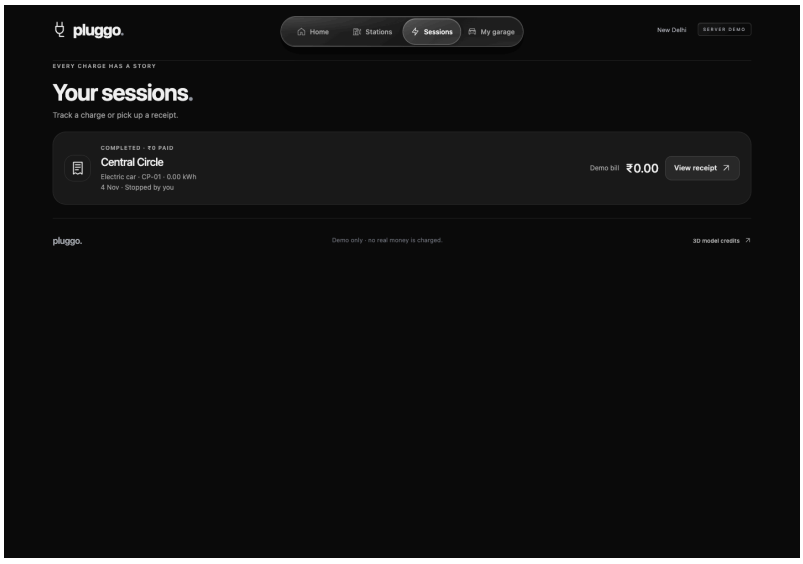


Figure 4. Saved project screenshot of the customer session-history interface.

APPENDIX B: CODE EXTRACTS AND PSEUDOCODE

B.1 Actual Pricing Overload Declarations

Extracted from PricingEngine in include/ev_system.hpp:

```
double calculateCost(double energyKWh,
                     double flatRatePerKWh) const;
double calculateCost(double peakEnergyKWh,
                     double offPeakEnergyKWh,
                     double peakRate, double offPeakRate) const;
double calculateCost(double energyKWh, double baseRatePerKWh,
                     double solarDiscountPercent) const;
```

B.2 Actual Template Queue Comparator

The comparator uses exact scores and an insertion sequence for equal priorities:

```
bool operator()(const Node& left, const Node& right) const {
    if (left.score != right.score) {
        return left.score < right.score;
    }
    return left.sequence > right.sequence;
}
```

B.3 Deadline Scheduler — Condensed Pseudocode

```
validate target, available time and charging efficiency
energy = capacity * (target - currentSoC) / 100
if energy is effectively zero: return satisfied plan
requiredPower = energy / (hours * efficiency)
for each available compatible charger:
    usablePower = min(chargerPower, vehicleLimit)
    track smallest sufficient and fastest candidate
select sufficient candidate, otherwise fastest candidate
if no candidate: return infeasible plan with no charger
assign requiredPower if feasible, otherwise usablePower
return charger, energy, power, duration and feasibility
```

B.4 Reproducing the Domain Verification

```
clang++ -std=c++17 -Wall -Wextra -Wpedantic -Iinclude   tests/ev_tests.cpp src/
ev_system.cpp -o /tmp/ev-report-tests
/tmp/ev-report-tests
# Observed output: All EV feature checks passed.
```

End of Report